Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Dhruv Jangid **1BF24EC055**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

MODULE 01: ARITHMETIC LOGIC UNIT (ALU)

1.1 Design Overview & Architecture

A 4-bit Arithmetic Logic Unit (ALU) is a combinational datapath circuit that executes arithmetic and bitwise logical operations on two 4-bit unsigned/two's-complement inputs (A and B) based on a 3-bit operation select bus (ALU_Sel). The module outputs a 4-bit result (ALU_Out) alongside a Carry/Borrow flag (CarryOut) for arithmetic operations.

- **Port Specifications:** Inputs: A[3:0], B[3:0], ALU_Sel[2:0] | Outputs: ALU_Out[3:0], CarryOut.
- **Synthesizable Construct:** Implemented using a combinational always @(*) block and a full 8-way case statement to prevent unintended latch inference.

1.2 Functional Operation Table

ALU_Sel[2:0]	Operation Name	RTL Expression	CarryOut Behavior
3'b000	Arithmetic Addition	{CarryOut, ALU_Out} = A + B	1 if (A + B) > 15, else 0
3'b001	Arithmetic Subtraction	{CarryOut, ALU_Out} = A - B	Borrow flag if A < B
3'b010	Bitwise AND	ALU_Out = A & B	0
3'b011	Bitwise OR	ALU_Out = A | B	0
3'b100	Bitwise XOR	ALU_Out = A ^ B	0
3'b101	Bitwise NOT (Complement)	ALU_Out = ~A	0
3'b110	Logical Shift Left	ALU_Out = A << 1	0
3'b111	Logical Shift Right	ALU_Out = A >> 1	0

1.3 Verilog HDL Code (alu_4bit.v)

```
module alu_4bit (
    input  [3:0] A,
    input  [3:0] B,
    input  [2:0] ALU_Sel,
    output reg [3:0] ALU_Out,
    output reg CarryOut
);
    always @(*) begin
        CarryOut = 1'b0;
        case (ALU_Sel)
            3'b000: {CarryOut, ALU_Out} = A + B;       // Addition
            3'b001: {CarryOut, ALU_Out} = A - B;       // Subtraction
            3'b010: ALU_Out = A & B;                   // Bitwise AND
            3'b011: ALU_Out = A | B;                   // Bitwise OR
            3'b100: ALU_Out = A ^ B;                   // Bitwise XOR
            3'b101: ALU_Out = ~A;                      // Bitwise NOT (Invert A)
            3'b110: ALU_Out = A << 1;                  // Logical Shift Left
            3'b111: ALU_Out = A >> 1;                  // Logical Shift Right
```

```
            default: ALU_Out = 4'b0000;
        endcase
    end
endmodule
```

1.4 Testbench Code (tb_alu_4bit.v)

```
`timescale 1ns / 1ps
module tb_alu_4bit;
    reg  [3:0] A, B;
    reg  [2:0] ALU_Sel;
    wire [3:0] ALU_Out;
    wire CarryOut;
    integer i;

    alu_4bit uut (
        .A(A),
        .B(B),
        .ALU_Sel(ALU_Sel),
        .ALU_Out(ALU_Out),
        .CarryOut(CarryOut)
    );

    initial begin
        $dumpfile("alu_4bit.vcd");
        $dumpvars(0, tb_alu_4bit);
        $monitor("Time=%0t | A=%b B=%b | Sel=%b | Out=%b CarryOut=%b",
                 $time, A, B, ALU_Sel, ALU_Out, CarryOut);

        A = 4'b1010; B = 4'b0101;
        for (i = 0; i < 8; i = i + 1) begin
            ALU_Sel = i[2:0];
            #10;
        end
        // Test addition overflow
        A = 4'b1111; B = 4'b0010; ALU_Sel = 3'b000; #10;
        $finish;
    end
endmodule
```

MODULE 02: 4-BIT MAGNITUDE COMPARATOR

2.1 Design Overview & Architecture

A 4-bit Magnitude Comparator compares two 4-bit unsigned binary numbers A[3:0] and B[3:0] and asserts exactly one of three mutually exclusive active-high outputs: A_gt_B (A > B), A_eq_B (A == B), or A_lt_B (A < B). In hardware, equality is determined by bitwise XNOR reduction, while relative magnitude is resolved from MSB to LSB.

2.2 Truth & Condition Table

Input Condition	A_gt_B	A_eq_B	A_lt_B	Example Vector (A, B)
A > B	1	0	0	A = 4'b1000 (8), B = 4'b0011 (3)
A == B	0	1	0	A = 4'b0101 (5), B = 4'b0101 (5)
A < B	0	0	1	A = 4'b0010 (2), B = 4'b1100 (12)

2.3 Verilog HDL Code (comparator_4bit.v)

```
module comparator_4bit (
    input  [3:0] A,
    input  [3:0] B,
    output reg A_gt_B,
    output reg A_eq_B,
    output reg A_lt_B
);
    always @(*) begin
        A_gt_B = 1'b0;
        A_eq_B = 1'b0;
        A_lt_B = 1'b0;
        if (A > B)
            A_gt_B = 1'b1;
        else if (A < B)
            A_lt_B = 1'b1;
        else
            A_eq_B = 1'b1;
    end
endmodule
```

2.4 Testbench Code (tb_comparator_4bit.v)

```
`timescale 1ns / 1ps
module tb_comparator_4bit;
    reg  [3:0] A, B;
    wire A_gt_B, A_eq_B, A_lt_B;

    comparator_4bit uut (
        .A(A), .B(B),
        .A_gt_B(A_gt_B), .A_eq_B(A_eq_B), .A_lt_B(A_lt_B)
    );

    initial begin
        $dumpfile("comparator_4bit.vcd");
        $dumpvars(0, tb_comparator_4bit);
        $monitor("Time=%0t | A=%d (%b) B=%d (%b) | GT=%b EQ=%b LT=%b",
                 $time, A, A, B, B, A_gt_B, A_eq_B, A_lt_B);

        A = 4'd8;  B = 4'd3;  #10; // A > B
        A = 4'd5;  B = 4'd5;  #10; // A == B
        A = 4'd2;  B = 4'd12; #10; // A < B
```

```
        A = 4'd15; B = 4'd0;  #10; // Boundary test
        $finish;
    end
endmodule
```

MODULE 03: 4-BIT PARITY GENERATOR AND CHECKER

3.1 Design Overview & Architecture

Parity generators and checkers are fundamental single-bit error detection circuits in digital communication links. At the transmitter, the 4-bit message (data_in[3:0]) is passed through an XOR reduction tree to compute the Even Parity bit (even_parity_out) and an XNOR tree for the Odd Parity bit (odd_parity_out). At the receiver, the 4-bit data and received parity bit (parity_in) are XOR-reduced to produce parity_error (0 = valid even parity transmission; 1 = single-bit error detected).

- **Boolean Equations:** P_even = D[3] ^ D[2] ^ D[1] ^ D[0] | P_odd = ~(P_even) | Error_even = D[3] ^ D[2] ^ D[1] ^ D[0] ^ P_in

3.2 Sample Parity Verification Table

data_in[3:0]	Number of 1s	even_parity_out	odd_parity_out	parity_in (Rx)	parity_error
4'b1011	3 (Odd)	1	0	1 (Valid)	0 (No Error)
4'b1011	3 (Odd)	1	0	0 (Corrupted)	1 (Error Detected)
4'b1100	2 (Even)	0	1	0 (Valid)	0 (No Error)
4'b0000	0 (Even)	0	1	1 (Corrupted)	1 (Error Detected)

3.3 Verilog HDL Code (parity_gen_check_4bit.v)

```
module parity_gen_check_4bit (
    input  [3:0] data_in,
    input        parity_in,
    output       even_parity_out,
    output       odd_parity_out,
    output       parity_error
);
    // Parity Generation using reduction operators
    assign even_parity_out = ^data_in;                  // XOR reduction
    assign odd_parity_out  = ~(^data_in);               // XNOR reduction

    // Even Parity Error Checker (0 = No Error, 1 = Error)
    assign parity_error    = ^{data_in, parity_in};
endmodule
```

3.4 Testbench Code (tb_parity_gen_check_4bit.v)

```
`timescale 1ns / 1ps
module tb_parity_gen_check_4bit;
    reg  [3:0] data_in;
    reg        parity_in;
    wire       even_parity_out, odd_parity_out, parity_error;

    parity_gen_check_4bit uut (
        .data_in(data_in),
        .parity_in(parity_in),
        .even_parity_out(even_parity_out),
        .odd_parity_out(odd_parity_out),
        .parity_error(parity_error)
    );

    initial begin
        $dumpfile("parity_gen_check.vcd");
        $dumpvars(0, tb_parity_gen_check_4bit);
        $monitor("Time=%0t | Data=%b ParIn=%b | EvenGen=%b OddGen=%b | Error=%b",
                 $time, data_in, parity_in, even_parity_out, odd_parity_out, parity_error);

        data_in = 4'b1011; parity_in = 1'b1; #10; // Valid Even Parity (Error = 0)
        data_in = 4'b1011; parity_in = 1'b0; #10; // Corrupted Bit (Error = 1)
        data_in = 4'b1100; parity_in = 1'b0; #10; // Valid Even Parity (Error = 0)
        data_in = 4'b0111; parity_in = 1'b0; #10; // Corrupted Bit (Error = 1)
        $finish;
    end
endmodule
```

MODULE 04: 4-BIT BARREL SHIFTER

4.1 Design Overview & Architecture

Unlike sequential shift registers that require N clock cycles to shift data by N positions, a combinational Barrel Shifter shifts or rotates an entire multi-bit word by any specified number of positions within a single combinational propagation delay using an array of multiplexers. This 4-bit Barrel Shifter performs a circular left rotation (ROL) on data_in[3:0] controlled by shift_amt[1:0].

4.2 Multiplexer Routing Table

shift_amt[1:0]	Rotation Count	Output Mapping {MSB ... LSB}	Example (data_in = 4'b1001)
2'b00	No Rotation (0 bits)	{data_in[3], data_in[2], data_in[1], data_in[0]}	4'b1001
2'b01	Rotate Left by 1 bit	{data_in[2], data_in[1], data_in[0], data_in[3]}	4'b0011
2'b10	Rotate Left by 2 bits	{data_in[1], data_in[0], data_in[3], data_in[2]}	4'b0110

2'b11	Rotate Left by 3 bits	{data_in[0], data_in[3], data_in[2], data_in[1]}	4'b1100

4.3 Verilog HDL Code (barrel_shifter_4bit.v)

```
module barrel_shifter_4bit (
    input  [3:0] data_in,
    input  [1:0] shift_amt,
    output reg [3:0] data_out
);
    always @(*) begin
        case (shift_amt)
            2'b00: data_out = data_in;
            2'b01: data_out = {data_in[2:0], data_in[3]};
            2'b10: data_out = {data_in[1:0], data_in[3:2]};
            2'b11: data_out = {data_in[0],   data_in[3:1]};
            default: data_out = data_in;
        endcase
    end
endmodule
```

4.4 Testbench Code (tb_barrel_shifter_4bit.v)

```
`timescale 1ns / 1ps
module tb_barrel_shifter_4bit;
    reg  [3:0] data_in;
    reg  [1:0] shift_amt;
    wire [3:0] data_out;

    barrel_shifter_4bit uut (
        .data_in(data_in),
        .shift_amt(shift_amt),
        .data_out(data_out)
    );

    initial begin
        $dumpfile("barrel_shifter.vcd");
        $dumpvars(0, tb_barrel_shifter_4bit);
        $monitor("Time=%0t | data_in=%b | shift_amt=%d | data_out=%b",
                 $time, data_in, shift_amt, data_out);

        data_in = 4'b1001;
        shift_amt = 2'b00; #10;
        shift_amt = 2'b01; #10;
        shift_amt = 2'b10; #10;
        shift_amt = 2'b11; #10;
        $finish;
    end
endmodule
```

MODULE 05: PROGRAMMABLE TIMER MODULE

5.1 Design Overview & Architecture

The 4-bit Programmable Timer Module is a synchronous countdown timer used in embedded controllers and watchdog circuits. When 'load' is asserted, the counter initializes to a user-defined preset value (load_val[3:0]). When 'en' is high and the counter is non-zero, it decrements on each rising clock edge. Upon reaching 4'b0000, the module halts underflow and asserts an active-high 'timeout' flag.

- **Control Priority:** Asynchronous active-high Reset (rst) > Synchronous Parallel Load (load) > Count Enable (en).

5.2 Control Mode Summary

rst	load	en	count != 0	Register Next-State Action	timeout Flag
1	X	X	X	count <= 4'b0000 (Async Reset)	1
0	1	X	X	count <= load_val (Preset Load)	1 if load_val==0 else 0
0	0	1	Yes	count <= count - 1'b1 (Decrement)	0
0	0	1	No (0)	count <= 4'b0000 (Hold at Zero)	1 (Terminal Count)
0	0	0	X	count <= count (Pause Timer)	count == 0

5.3 Verilog HDL Code (timer_module.v)

```
module timer_module (
    input        clk,
    input        rst,
    input        load,
    input        en,
    input  [3:0] load_val,
    output reg [3:0] count,
    output       timeout
);
    assign timeout = (count == 4'b0000);

    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 4'b0000;
        else if (load)
            count <= load_val;
        else if (en && count != 4'b0000)
            count <= count - 1'b1;
    end
endmodule
```

5.4 Testbench Code (tb_timer_module.v)

```
`timescale 1ns / 1ps
module tb_timer_module;
```

```
    reg        clk, rst, load, en;
    reg  [3:0] load_val;
    wire [3:0] count;
    wire       timeout;

    timer_module uut (
        .clk(clk), .rst(rst), .load(load), .en(en),
        .load_val(load_val), .count(count), .timeout(timeout)
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("timer_module.vcd");
        $dumpvars(0, tb_timer_module);
        $monitor("Time=%0t | rst=%b load=%b en=%b | count=%d | timeout=%b",
                 $time, rst, load, en, count, timeout);

        clk = 0; rst = 1; load = 0; en = 0; load_val = 4'd5;
        #10 rst = 0; load = 1;       // Load preset value 5
        #10 load = 0; en = 1;        // Enable countdown
        #70 $finish;
    end
endmodule
```

MODULE 06: 4-BIT RING COUNTER

6.1 Design Overview & Architecture

A Ring Counter is a circular shift register in which the output of the least significant flip-flop (q[0]) is fed directly back to the input of the most significant flip-flop (q[3]). Upon reset, the register is pre-loaded with a single 'one-hot' active bit (4'b1000). A 4-bit Ring Counter cycles through N = 4 distinct one-hot states without requiring external decoding gates.

6.2 State Transition Table

Clock Cycle	q[3]	q[2]	q[1]	q[0]	Hex / One-Hot State
0 (Reset)	1	0	0	0	4'h8 (State S0)
1	0	1	0	0	4'h4 (State S1)
2	0	0	1	0	4'h2 (State S2)
3	0	0	0	1	4'h1 (State S3)
4 (Repeats)	1	0	0	0	4'h8 (State S0)

6.3 Verilog HDL Code (ring_counter_4bit.v)

```
module ring_counter_4bit (
```

```
    input            clk,
    input            rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b1000;          // Initialize one-hot seed on reset
        else
            q <= {q[0], q[3:1]};   // Circular right shift
    end
endmodule
```

6.4 Testbench Code (tb_ring_counter_4bit.v)

```
`timescale 1ns / 1ps
module tb_ring_counter_4bit;
    reg        clk, rst;
    wire [3:0] q;

    ring_counter_4bit uut (.clk(clk), .rst(rst), .q(q));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("ring_counter.vcd");
        $dumpvars(0, tb_ring_counter_4bit);
        $monitor("Time=%0t | rst=%b | Ring Output q=%b", $time, rst, q);

        clk = 0; rst = 1;
        #10 rst = 0;
        #60 $finish;
    end
endmodule
```

MODULE 07: 4-BIT JOHNSON (TWISTED-RING) COUNTER

7.1 Design Overview & Architecture

A Johnson Counter (also known as a Twisted-Ring or Mobius Counter) connects the complemented output of the last flip-flop (~q[0]) back to the serial input of the first flip-flop (q[3]). Starting from 4'b0000, an N-bit Johnson counter progresses through 2N = 8 unique states where only a single bit changes per clock transition (Gray-like glitch-free switching).

7.2 8-State Sequence Table

Clock Cycle	Binary Output q[3:0]	Feedback Bit (~q[0])	Decimal Equivalent
0 (Reset)	0000	1	0
1	1000	1	8

2	1100	1	12
3	1110	1	14
4	1111	0	15
5	0111	0	7
6	0011	0	3
7	0001	1	1

7.3 Verilog HDL Code (johnson_counter_4bit.v)

```
module johnson_counter_4bit (
    input            clk,
    input            rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= {~q[0], q[3:1]};  // Inverted feedback into MSB
    end
endmodule
```

7.4 Testbench Code (tb_johnson_counter_4bit.v)

```
`timescale 1ns / 1ps
module tb_johnson_counter_4bit;
    reg        clk, rst;
    wire [3:0] q;

    johnson_counter_4bit uut (.clk(clk), .rst(rst), .q(q));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("johnson_counter.vcd");
        $dumpvars(0, tb_johnson_counter_4bit);
        $monitor("Time=%0t | rst=%b | Johnson Output q=%b", $time, rst, q);

        clk = 0; rst = 1;
        #10 rst = 0;
        #90 $finish;
    end
endmodule
```

MODULE 08: SYNCHRONOUS AND ASYNCHRONOUS COUNTERS

8.1 Design Overview & Architectural Comparison

Binary counters step through the natural 4-bit binary sequence (0000 to 1111). In a Synchronous Counter, every flip-flop shares the primary system clock and updates simultaneously, yielding high maximum operating frequency. In an Asynchronous (Ripple) Counter, only q[0] receives the system clock, while each subsequent stage is clocked by the falling edge (negedge) of the preceding stage.

Feature	4-Bit Synchronous Counter	4-Bit Asynchronous (Ripple) Counter
Clocking Scheme	Common clk connected to all 4 flip-flops	q[0] clocked by clk; q[i] clocked by negedge q[i-1]
Propagation Delay	T_pd = T_cq + T_comb (Single stage delay)	T_pd = 4 x T_cq (Cumulative ripple delay)
Decoding Glitches	Minimal (simultaneous state transitions)	Prone to transient glitches during ripple settling
Hardware Complexity	Requires carry-chain combinational logic	Zero inter-stage combinational gates

8.2 Verilog HDL Code (sync_async_counters.v)

```
// 1. 4-Bit Synchronous Binary Up Counter
module sync_counter_4bit (
    input            clk,
    input            rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= q + 1'b1;
    end
endmodule

// 2. 4-Bit Asynchronous (Ripple) Binary Up Counter
module async_counter_4bit (
    input            clk,
    input            rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst)
        if (rst) q[0] <= 1'b0; else q[0] <= ~q[0];

    always @(negedge q[0] or posedge rst)
        if (rst) q[1] <= 1'b0; else q[1] <= ~q[1];

    always @(negedge q[1] or posedge rst)
        if (rst) q[2] <= 1'b0; else q[2] <= ~q[2];

    always @(negedge q[2] or posedge rst)
```

```
        if (rst) q[3] <= 1'b0; else q[3] <= ~q[3];
endmodule
```

8.3 Testbench Code (tb_counters_4bit.v)

```
`timescale 1ns / 1ps
module tb_counters_4bit;
    reg        clk, rst;
    wire [3:0] q_sync, q_async;

    sync_counter_4bit  uut_sync  (.clk(clk), .rst(rst), .q(q_sync));
    async_counter_4bit uut_async (.clk(clk), .rst(rst), .q(q_async));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("counters_4bit.vcd");
        $dumpvars(0, tb_counters_4bit);
        $monitor("Time=%0t | rst=%b | Sync_Count=%d (%b) | Async_Count=%d (%b)",
                 $time, rst, q_sync, q_sync, q_async, q_async);

        clk = 0; rst = 1;
        #10 rst = 0;
        #170 $finish;
    end
endmodule
```

MODULE 09: 4-BIT SHIFT REGISTER (SIPO / SISO)

9.1 Design Overview & Architecture

A 4-bit Shift Register consists of four cascaded D flip-flops sharing a common clock. On each rising clock edge, incoming serial data (serial_in) is shifted into the MSB (parallel_out[3]), while existing bits shift right toward the LSB (parallel_out[0]). Exposing all 4 flip-flop outputs provides Serial-In Parallel-Out (SIPO) functionality, while tapping parallel_out[0] provides Serial-In Serial-Out (SISO) functionality with a 4-cycle latency.

9.2 Shift Operation Trace (Input Bitstream: 1, 0, 1, 1)

Clock Edge	serial_in	parallel_out[3:0]	serial_out (parallel_out[0])
Reset	0	0000	0
Cycle 1	1	1000	0
Cycle 2	0	0100	0
Cycle 3	1	1010	0
Cycle 4	1	1101 (Full 4-bit frame aligned)	1

9.3 Verilog HDL Code (shift_register_4bit.v)

```
module shift_register_4bit (
    input            clk,
    input            rst,
    input            serial_in,
    output           serial_out,
    output reg [3:0] parallel_out
);
    assign serial_out = parallel_out[0];

    always @(posedge clk or posedge rst) begin
        if (rst)
            parallel_out <= 4'b0000;
        else
            parallel_out <= {serial_in, parallel_out[3:1]};
    end
endmodule
```

9.4 Testbench Code (tb_shift_register_4bit.v)

```
`timescale 1ns / 1ps
module tb_shift_register_4bit;
    reg        clk, rst, serial_in;
    wire       serial_out;
    wire [3:0] parallel_out;

    shift_register_4bit uut (
        .clk(clk), .rst(rst), .serial_in(serial_in),
        .serial_out(serial_out), .parallel_out(parallel_out)
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("shift_register.vcd");
        $dumpvars(0, tb_shift_register_4bit);
        $monitor("Time=%0t | rst=%b sin=%b | parallel_out=%b | sout=%b",
                 $time, rst, serial_in, parallel_out, serial_out);

        clk = 0; rst = 1; serial_in = 0;
        #10 rst = 0;
        serial_in = 1; #10;
        serial_in = 0; #10;
        serial_in = 1; #10;
        serial_in = 1; #10;
        serial_in = 0; #40;
        $finish;
    end
endmodule
```

MODULE 10: PARAMETERIZED DIVIDE-BY-N COUNTER

10.1 Design Overview & Architecture

A Divide-by-N (Modulo-N) Counter counts from 0 up to (N - 1) and synchronously wraps back to 0 on the N-th clock edge. It generates a periodic enable/divided clock pulse (div_clk) at a frequency of f_out = f_clk / N. Using Verilog parameters (N = 5, WIDTH = 3), the module can be instantiated for any arbitrary division ratio.

10.2 Modulo-5 State Table (N = 5)

Clock Cycle	Current Count	Next Count	div_clk Output
0 (Reset)	0	1	0
1	1	2	0
2	2	3	0
3	3	4 (N - 1)	0
4	4 (Terminal Count)	0 (Wrap Around)	1 (Pulse Asserted)

10.3 Verilog HDL Code (divide_by_n_counter.v)

```
module divide_by_n_counter #(
    parameter N = 5,
    parameter WIDTH = 3
)(
    input                  clk,
    input                  rst,
    output reg [WIDTH-1:0] count,
    output reg             div_clk
);
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count   <= 0;
            div_clk <= 1'b0;
        end else if (count == N - 1) begin
            count   <= 0;
            div_clk <= 1'b1;
        end else begin
            count   <= count + 1'b1;
            div_clk <= 1'b0;
        end
    end
endmodule
```

10.4 Testbench Code (tb_divide_by_n_counter.v)

```
`timescale 1ns / 1ps
module tb_divide_by_n_counter;
    reg        clk, rst;
```

```
    wire [2:0] count;
    wire       div_clk;

    divide_by_n_counter #(.N(5), .WIDTH(3)) uut (
        .clk(clk), .rst(rst), .count(count), .div_clk(div_clk)
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("divide_by_n.vcd");
        $dumpvars(0, tb_divide_by_n_counter);
        $monitor("Time=%0t | rst=%b | count=%d | div_clk=%b",
                 $time, rst, count, div_clk);

        clk = 0; rst = 1;
        #10 rst = 0;
        #120 $finish;
    end
endmodule
```

MODULE 11: MEALY AND MOORE FSMS (101 SEQUENCE DETECTOR)

11.1 Design Overview & Architectural Comparison

To illustrate both Finite State Machine paradigms, an overlapping '101' serial bit sequence detector is designed in both Moore and Mealy architectures:

- **Moore FSM (4 States - S0, S1, S2, S3):** The output dout depends solely on the present state (dout = 1 only in state S3). Output changes synchronously with the clock edge.
- **Mealy FSM (3 States - S0, S1, S2):** The output dout depends on both the present state and the current input (dout = 1 when in state S2 and din == 1). Requires one fewer state and responds one clock cycle earlier.

11.2 State Transition Table (Moore vs. Mealy)

FSM Type	Present State	Meaning	Next State (din=0)	Next State (din=1)	Output (dout)
Moore	S0 (2'b00)	Reset / Idle	S0	S1	0
Moore	S1 (2'b01)	'1' Detected	S2	S1	0
Moore	S2 (2'b10)	'10' Detected	S0	S3	0
Moore	S3 (2'b11)	'101' Detected	S2	S1	1 (State-Only)
Mealy	S0 (2'b00)	Reset / Idle	S0	S1	din=0: 0 | din=1: 0
Mealy	S1 (2'b01)	'1' Detected	S2	S1	din=0: 0 | din=1: 0
Mealy	S2 (2'b10)	'10' Detected	S0	S1	din=0: 0 | din=1: 1

11.3 Verilog HDL Code (fsm_moore_mealy_101.v)

```
// 1. Moore FSM - Overlapping "101" Sequence Detector
module fsm_moore_101 (
    input      clk,
    input      rst,
    input      din,
    output reg dout
);
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst) begin
        if (rst) state <= S0;
        else     state <= next_state;
    end

    always @(*) begin
        case (state)
            S0: next_state = din ? S1 : S0;
            S1: next_state = din ? S1 : S2;
            S2: next_state = din ? S3 : S0;
            S3: next_state = din ? S1 : S2;
            default: next_state = S0;
        endcase
    end

    always @(*) begin
        dout = (state == S3) ? 1'b1 : 1'b0;
    end
endmodule

// 2. Mealy FSM - Overlapping "101" Sequence Detector
module fsm_mealy_101 (
    input      clk,
    input      rst,
    input      din,
    output reg dout
);
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst) begin
        if (rst) state <= S0;
        else     state <= next_state;
    end

    always @(*) begin
        dout = 1'b0;
        case (state)
            S0: next_state = din ? S1 : S0;
            S1: next_state = din ? S1 : S2;
            S2: begin
                next_state = din ? S1 : S0;
                dout       = din ? 1'b1 : 1'b0;
            end
            default: next_state = S0;
```

```
        endcase
    end
endmodule
```

11.4 Testbench Code (tb_fsm_101.v)

```
`timescale 1ns / 1ps
module tb_fsm_101;
    reg  clk, rst, din;
    wire dout_moore, dout_mealy;

    fsm_moore_101 uut_moore (.clk(clk), .rst(rst), .din(din), .dout(dout_moore));
    fsm_mealy_101 uut_mealy (.clk(clk), .rst(rst), .din(din), .dout(dout_mealy));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("fsm_101.vcd");
        $dumpvars(0, tb_fsm_101);
        $monitor("Time=%0t | rst=%b din=%b | Moore_Out=%b | Mealy_Out=%b",
                 $time, rst, din, dout_moore, dout_mealy);

        clk = 0; rst = 1; din = 0;
        #10 rst = 0;
        // Overlapping sequence: 1 -> 0 -> 1 -> 0 -> 1
        din = 1; #10;
        din = 0; #10;
        din = 1; #10;
        din = 0; #10;
        din = 1; #10;
        din = 0; #20;
        $finish;
    end
endmodule
```

MODULE 12: RANDOM SEQUENCE GENERATOR (4-BIT LFSR)

12.1 Design Overview & Architecture

A Linear Feedback Shift Register (LFSR) generates a deterministic, maximal-length pseudo-random binary sequence (PRBS) commonly used in Built-In Self-Test (BIST), cryptography, and data scramblers. By selecting taps corresponding to the primitive polynomial P(x) = x^4 + x^3 + 1 (feedback = q[3] ^ q[2]) and initializing the register with a non-zero seed (4'b1111), the 4-bit LFSR cycles through all 2^4 - 1 = 15 non-zero states before repeating.

12.2 Maximal-Length 15-State Pseudo-Random Sequence

Cycle	q[3:0] (Bin / Hex)	Feedback (q[3]^q[2])	Cycle	q[3:0] (Bin / Hex)	Feedback (q[3]^q[2])
0 (Seed)	1111 (F)	0	8	0101 (5)	1

1	1110 (E)	0	9	1011 (B)	1
2	1100 (C)	0	10	0111 (7)	1
3	1000 (8)	1	11	1111 (Wraps at 15)	-
4	0001 (1)	0	Note	All-Zero State (0000)	Excluded (Lock-up state)

12.3 Verilog HDL Code (lfsr_4bit.v)

```
module lfsr_4bit (
    input            clk,
    input            rst,
    output reg [3:0] q
);
    wire feedback;
    // Primitive polynomial: x^4 + x^3 + 1
    assign feedback = q[3] ^ q[2];

    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b1111;          // Must initialize to non-zero seed
        else
            q <= {q[2:0], feedback};
    end
endmodule
```

12.4 Testbench Code (tb_lfsr_4bit.v)

```
`timescale 1ns / 1ps
module tb_lfsr_4bit;
    reg        clk, rst;
    wire [3:0] q;

    lfsr_4bit uut (.clk(clk), .rst(rst), .q(q));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("lfsr_4bit.vcd");
        $dumpvars(0, tb_lfsr_4bit);
        $monitor("Time=%0t | rst=%b | Pseudo-Random Output q=%b (%d)",
                 $time, rst, q, q);

        clk = 0; rst = 1;
        #10 rst = 0;
        #160 $finish;
    end
endmodule
```